

CSCI 423

Introduction to Algorithms



DR. RAAFAT ELFOULY



Chapter 4, Part I

SORTING ALGORITHMS

Chapter Outline

3

- Bubble sort
- Insertion sort
- Shellsort
- Radix sort
- Heapsort
- Merge sort
- Quicksort

Prerequisites

4

- Before beginning this chapter, you should be able to:
 - Read and create iterative and algorithms
 - Use summations and probabilities presented in Chapter 1
 - Solve recurrence relations in Chapter 2
 - Describe growth rates and order

Goals

5

- At the end of this chapter you should be able to:
 - Explain insertion sort and its analysis
 - Explain bubble sort and its analysis
 - Explain shellsort and its analysis
 - Explain radix sort and its analysis

Bubble Sort

6

- If we compare pairs of adjacent elements and none are out of order, the list is sorted
- If any are out of order, we must have to swap them to get an ordered list
- Bubble sort will make passes through the list swapping any adjacent elements that are out of order

Bubble Sort

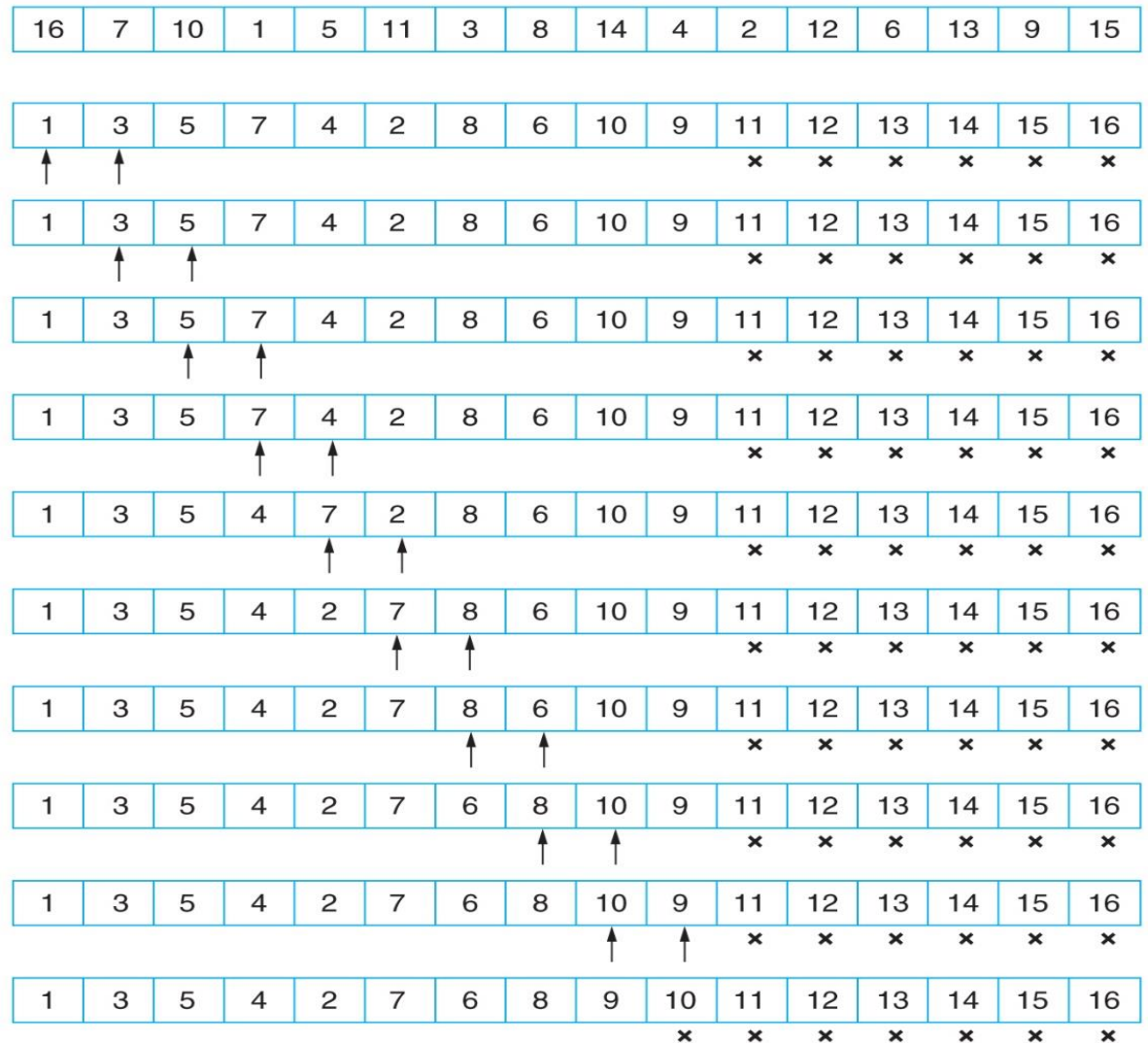
7

- After the first pass, we know that the largest element must be in the correct place
- After the second pass, we know that the second largest element must be in the correct place
- Because of this, we can shorten each successive pass of the comparison loop

Bubble Sort Example

8

■ **FIGURE 4.2**
Bubble sort





- <https://visualgo.net/bn/sorting>



- Try to write algorithm's pseudocode

Bubble Sort Algorithm

11

```
numberOfPairs = N
swappedElements = true
while swappedElements do
  numberOfPairs = numberOfPairs - 1
  swappedElements = false
  for i = 1 to numberOfPairs do
    if list[ i ] > list[ i + 1 ] then
      Swap( list[i], list[i + 1] )
      swappedElements = true
    end if
  end for
end while
```



- Trace the algorithm for the list:
- 4, 8, 3

Bubble Sort Algorithm

13

```
numberOfPairs = N
swappedElements = true
while swappedElements do
  numberOfPairs = numberOfPairs - 1
  swappedElements = false
  for i = 1 to numberOfPairs do
    if list[ i ] > list[ i + 1 ] then
      Swap( list[i], list[i + 1] )
      swappedElements = true
    end if
  end for
end while
```

Best-Case Analysis

14

- If the elements start in sorted order, the for loop will compare the adjacent pairs but not make any changes
- So the `swappedElements` variable will still be false and the `while` loop is only done once
- There are $N - 1$ comparisons in the best case

Worst-Case Analysis

15

- If in the best case the `while` loop is done once, in the worst case the `while` loop must be done as many times as possible
- Each pass of the `for` loop must make at least one swap of the elements
- The number of comparisons will be:

$$W(N) = \sum_{i=N-1}^1 i = \frac{(N-1) * N}{2} = O(N^2)$$

Average-Case Analysis

16

- We can potentially stop after any of the (at most) $N - 1$ passes of the `for` loop
- This means that we have $N - 1$ possibilities and the average case is given by

$$A(N) = \frac{1}{N - 1} \sum_{i=1}^{N-1} C(i)$$

- where $C(i)$ is the work done in the first i passes

Average-Case Analysis

17

- On the first pass, we do $N - 1$ comparisons
- On the second pass, we do $N - 2$ comparisons
- The number of comparisons in the first i passes, in other words $C(i)$, is given by:

$$C(i) = \sum_{j=N-1}^{N-i} j = N * i - \frac{i^2 + i}{2}$$



- $(N-i) + (N-i-1) + (N-i-2) + \dots + (N-1)$
- i terms
- First part $N*i$
- Second part : $\sum_{j=1}^i j = \frac{i^2 + i}{2}$
- Hence we get work done at stage i

$$C(i) = \sum_{j=N-1}^{N-i} j = N*i - \frac{i^2 + i}{2}$$

Average-Case Analysis

19

- Putting the equation for $C(i)$ into the equation for $A(N)$ we get:

$$\begin{aligned} A(N) &= \frac{1}{N-1} \sum_{i=1}^{N-1} \left(N * i - \frac{i^2 + i}{2} \right) \\ &= \frac{2N^2 - N}{6} \\ &= O(N^2) \end{aligned}$$



- Bubble sort in place? Needs extra memory?

Insertion Sort

21

- Adding a new element to a sorted list will keep the list sorted if the element is inserted in the correct place
- A one element list is sorted
- Inserting a second element in the proper place keeps the list sorted
- This is repeated until all the elements have been inserted into the sorted part of the list

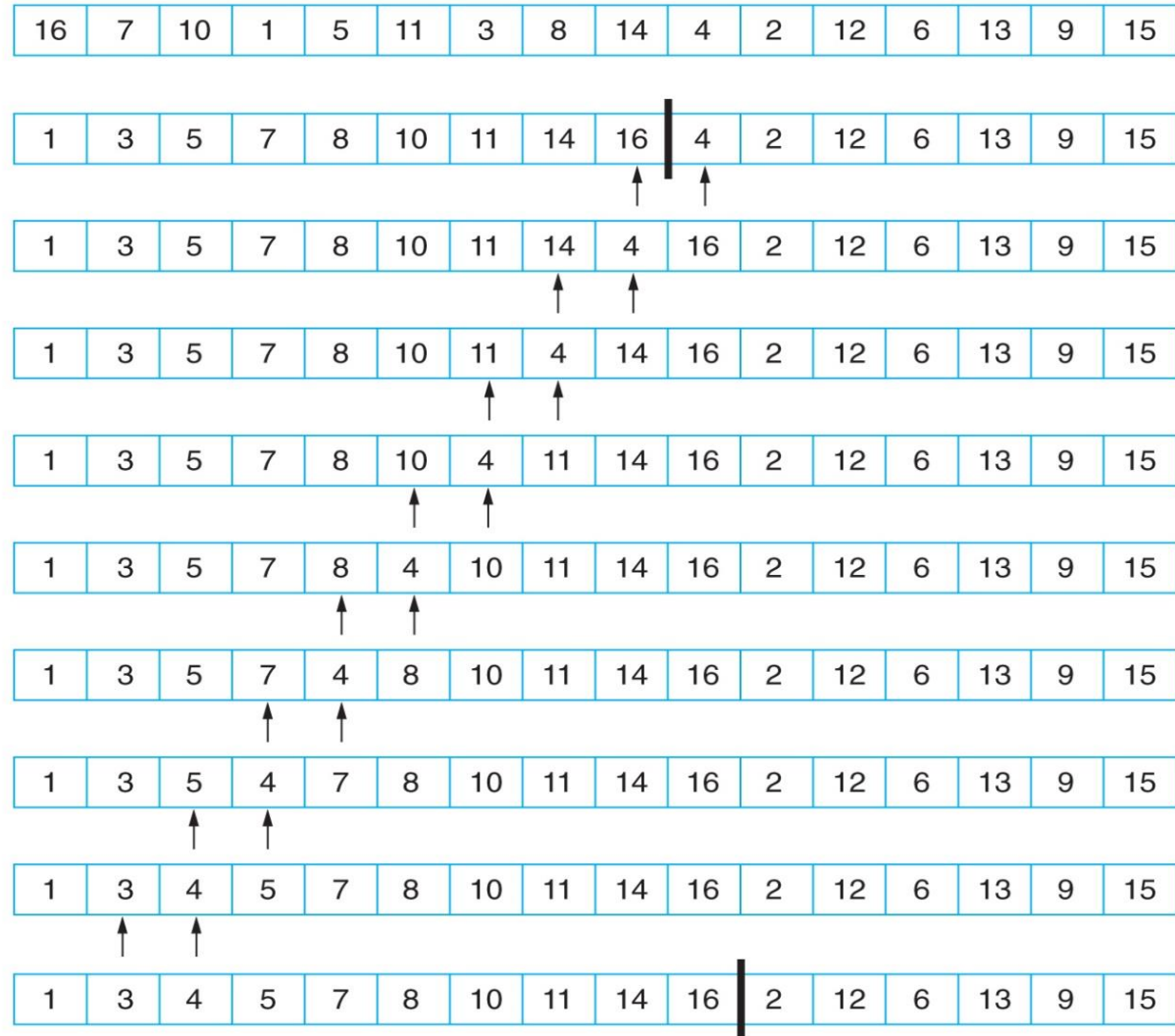


- <https://visualgo.net/bn/sorting>

Insertion Sort Example

23

■ **FIGURE 4.1**
Insertion sort





- Try to write algorithm's pseudocode

Insertion Sort Algorithm

25

```
for i = 2 to N do
    newElement = list[ i ]
    location = i - 1
    while (location ≥ 1) and
        (list[ location ] > newElement) do
        list[ location + 1 ] = list[ location ]
        location = location - 1
    end while
    list[ location + 1 ] = newElement
end for
```

Insertion Sort

26

Trace this Example: Sort 6, 4, 1, 8, 5

```
6 | 4  1  8  5
4  6 | 1  8  5
1  4  6 | 8  5
1  4  6  8 | 5
1  4  5  6  8
```

Worst-Case Analysis

27

Will happen if the list is in reverse order

- The outer loop is always done $N - 1$ times
- The inner loop does the most work when the next element is smaller than the past elements
- On each pass, the next element is compared to all earlier elements, giving:

$$W(N) = \sum_{i=1}^{N-1} i = \frac{(N-1) * N}{2} = O(N^2)$$

Average-Case Analysis

28

- There are $i + 1$ places where the i^{th} element can be added
- If it goes in the last location, we do one comparison
- If it goes in the second last location, we do two comparisons
- If it goes in the first or second location, we do i comparisons

Average-Case Analysis

29

- The average number of comparisons to insert the i^{th} element is:

$$A_i = \frac{1}{i+1} \left[i + \sum_{p=1}^i p \right] = \frac{i}{2} + 1 - \frac{1}{i+1}$$

- We now apply this for each of the algorithm's passes:

$$A(N) = \sum_{i=1}^{N-1} A_i \approx \frac{N^2}{4} = O(N^2)$$

Analysis of Insertion Sort

30

- Time efficiency

$$C_{worst}(n) = n(n-1)/2 \in \Theta(n^2)$$

$$C_{avg}(n) \approx n^2/4 \in \Theta(n^2)$$

$$C_{best}(n) = n - 1 \in \Theta(n) \text{ (also fast on almost sorted arrays)}$$

- Space efficiency: in-place
- Stability: yes
- Best elementary sorting algorithm overall
- What about Binary search would it help?



- Thank you